

Reading Material

Set Operations

Set operations **UNION**, **INTERSECT**, and **EXCEPT**:

- Each of the above operations automatically eliminates duplicates
- **UNION**
 - Used to combine the result of two or more sql SELECT queries.
 - In this operation, all the number of datatypes and columns must be the same in both the tables on which the **UNION** operation is being applied.
 - The union operation eliminates the duplicate rows from its resultset.
- **INTERSECT**
 - Used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
 - In this operation, the number of datatype and columns must be the same.
 - It has no duplicates, and it arranges the data in ascending order by default.
- **EXCEPT**
 - Used to combine two result sets and returns the data from the first result set, which is not present in the second result set.

Examples:

1. Find courses that ran in Fall 2017 or in Spring 2018.

```
(select course_id from section where sem = 'Fall' and year = 2017)
union
(select course_id from section where sem = 'Spring' and year = 2018)
```

2. Find courses that ran in Fall 2017 and in Spring 2018

```
(select course_id from section where sem = 'Fall' and year = 2017)
intersect
(select course_id from section where sem = 'Spring' and year = 2018)
```

3. Find courses that ran in Fall 2017 but not in Spring 2018

```
(select course_id from section where sem = 'Fall' and year = 2017)
except
(select course_id from section where sem = 'Spring' and year = 2018)
```

To retain all duplicates use the

- **UNION ALL**
- **INTERSECT ALL**
- **EXCEPT ALL**

Cross Product in SQL

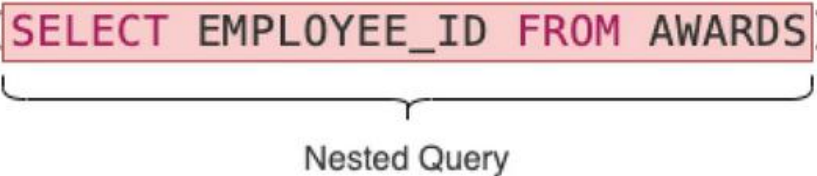
The cross product (or cartesian product) of two tables is a combination of every row in the first table with every row in the second table. In SQL, you can perform a cross product by using the CROSS JOIN keyword.

A	B		A x B
n	c		n c
1	x	SELECT * FROM A CROSS JOIN B	1 x
2	y		1 y
3	z		1 z
			2 x
			2 y
			2 z
			3 x
			3 y
			3 z

Nested Queries

In nested queries, a **query is written inside a query**. The result of inner query is used in execution of outer query.

```
SELECT ID, NAME FROM EMPLOYEES
WHERE ID IN (SELECT EMPLOYEE_ID FROM AWARDS)
```



Nested Query

Nested queries in SQL can be classified into two different types:

1. Independent Nested Queries

Here, the execution order is **from the innermost query to the outer query**. An outer query won't be executed until its inner query completes its execution. The result of the inner query is used by the outer query.

Operators such as **IN**, **NOT IN**, **ALL**, and **ANY** are used to write independent nested queries.

- The **IN** operator checks if a column value in the outer query's result is **present** in the inner query's result. The final result will have rows that satisfy the **IN** condition.
 - Here's the syntax of the **IN** operator:
value IN (value1, value2, value3,...)
 - In case the list has many values, you need to construct a very long statement with multiple **or** operators. Hence, the **in** operator allows you to shorten the query and make it more readable.
- The **NOT IN** operator checks if a column value in the outer query's result is **not present** in the inner query's result. The final result will have rows that satisfy the **NOT IN** condition.
 - The **NOT IN** operator returns 1 if the value doesn't equal any value in the list. Otherwise, it returns 0.
- The **ALL** operator compares a value of the outer query's result with **all the values** of the inner query's result and returns the row if it matches all the values.
- The **ANY** operator compares a value of the outer query's result with all the inner query's result values and returns the row if there is a match with **any value**.

Lets demonstrate this for the following structure of Database :

```
CREATE TABLE employee (  
    id NUMBER PRIMARY KEY,  
    name VARCHAR2(100) NOT NULL,  
    salary NUMBER NOT NULL,  
    role VARCHAR2(100) NOT NULL  
);
```

```
CREATE TABLE awards(  
    id NUMBER PRIMARY KEY,  
    employee_id NUMBER NOT NULL,  
    award_date DATE NOT NULL  
);
```

Example :

1. IN : Select all employees who won an award.

```
SELECT id, name FROM employees  
    WHERE id IN (SELECT employee_id FROM awards);
```

2. NOT IN : Select all employees who never won an award.

```
SELECT id, name FROM employees  
    WHERE id NOT IN (SELECT employee_id FROM awards);
```

3. ALL : Select all Developers who earn more than all the Managers

```
SELECT * FROM employees  
    WHERE role = 'Developer'  
    AND salary > ALL (SELECT salary FROM employees WHERE role = 'Manager'  
);
```

4. ANY : Select all Developers who earn more than any Manager

```
SELECT * FROM employees
WHERE role = 'Developer'
AND salary > ANY (
    SELECT salary FROM employees WHERE role = 'Manager'
);
```

2. Co-related Nested Queries

Here, the inner query uses the values from the outer query so that the **inner query is executed for every row processed by the outer query**. The co-related nested queries run slowly because the inner query is executed for every row of the outer query's result.

Example :

Select all employees whose salary is above the average salary of employees in their role.

```
SELECT * FROM employees emp1
WHERE salary > (
    SELECT AVG(salary) FROM employees emp2
    WHERE emp1.role = emp2.role
);
```

Examples on Set Membership :

1. Find courses offered in Fall 2017 and in Spring 2018

```
select distinct course_id from section
where semester = 'Fall' and year= 2017 and
    course_id in (select course_id
        from section where semester = 'Spring' and year= 2018);
```

2. Find courses offered in Fall 2017 but not in Spring 2018

```
select distinct course_id from section
    where semester = 'Fall' and year= 2017 and
        course_id not in (select course_id from section
            where semester = 'Spring' and year= 2018);
```

3. Name all instructors whose name is neither “Mozart” nor “Einstein”.

```
select distinct name from instructor
where name not in ('Mozart', 'Einstein')
```

4. Find the total number of (distinct) students who have taken course sections taught by the instructor with ID 10101

```
select count (distinct ID) from takes
where (course_id, sec_id, semester, year)
in (select course_id, sec_id, semester,
year from teaches where teaches.ID= 10101);
```

GROUP BY Clause

The **GROUP BY** Statement in SQL is used **to arrange identical data into groups** with the help of some aggregate functions (**COUNT()**, **MAX()**, **MIN()**, **SUM()**, **AVG()**).

If a particular column has the same values in different rows, then it will arrange these rows in a group. **GROUP BY** clause is used with the **SELECT** statement.

Example : Find the number of customers in each country

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country;
```

Group By with Multiple Columns

```
SELECT country, state, MIN(age) as min_age
FROM Persons
GROUP BY Country, State;
```

This groups all persons with similar country and state, and gives the minimum age of each group.

HAVING Clause

We can use HAVING clause to place conditions **to decide which group will be part of the final result-set**.

Also we can not use the aggregate functions like SUM(), COUNT() etc., with WHERE clause. So we have to use HAVING clause if we want to use any of these functions in the conditions.

ORDER BY Clause

ORDER BY statement in SQL is used **to sort the fetched data** in either ascending or descending according to one or more columns. By default, it sorts in ascending order, and we can use the keyword **DESC** to sort the data in descending order. (**ASC** for ascending order)

General Syntax :

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
ORDER BY column_name(s);
```

Example :

List the number of customers in each country, sorted high to low (Only include countries with more than 5 customers)

```
SELECT COUNT(CustomerID), Country FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) DESC;
```